Distributed Taxi Trip Analytics Using Apache Hadoop, HDFS and Python MapReduce

Mashami Pacifique
Big Data Essentials
ID: 101406

August 29, 2026

Contents

1 Introduction

This report documents the design and implementation of a Hadoop-based distributed analytics pipeline for NYC Yellow Taxi trip data, built using HDFS for storage and Python-based Hadoop Streaming for distributed MapReduce processing. The goal is to demonstrate how large-scale transportation data can be stored, cleaned, and analyzed across a Hadoop cluster, and to compare this approach against conventional single-machine processing with Pandas.

2 Business Problem

A transportation analytics company wants to understand taxi demand patterns, revenue distribution, popular routes, payment behavior, trip characteristics, and abnormal records within its trip data. Because the dataset spans millions of records, the analysis must use HDFS for distributed storage and Hadoop MapReduce for distributed processing, rather than a single-machine tool that would not scale to the full historical dataset.

3 Dataset Description

- **Source:** Official NYC Taxi & Limousine Commission (TLC) Trip Record Data (`https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page`).
- **Type used:** Yellow Taxi trip records.
- **Period covered:** January–May 2026 (5 monthly files), exceeding the assignment's minimum requirement of 2–3 months.
- **Original format:** Apache Parquet (as distributed by TLC).
- **Working format:** Converted to CSV using Python (pandas + pyarrow) prior to upload, since Hadoop Streaming operates naturally on line-oriented text data. Parquet is columnar and compressed, which is efficient for long-term storage and analytical query engines, but CSV's line-per-record structure maps directly onto the map-per-line model used by Hadoop Streaming, making it the more practical choice for this assignment.
- **Scale:** 18,999,282 raw data rows across the 5 files (18,999,287 lines including 5 header rows), comfortably exceeding the assignment's 5-million-record threshold.

Table 1: Raw dataset file sizes

File	Local (Parquet)	HDFS (CSV)
yellow_tripdata_2026-01	64.2 MB	379.5 MB
yellow_tripdata_2026-02	58.7 MB	346.0 MB
yellow_tripdata_2026-03	67.9 MB	405.0 MB
yellow_tripdata_2026-04	64.8 MB	393.9 MB
yellow_tripdata_2026-05	69.7 MB	419.2 MB
Total	**340 MB**	**1.9 GB**

3.1 Schema

Each record contains 20 comma-separated fields:

Idx	Field	Idx	Field
0	VendorID	10	fare_amount
1	tpep_pickup_datetime	11	extra
2	tpep_dropoff_datetime	12	mta_tax
3	passenger_count	13	tip_amount
4	trip_distance	14	tolls_amount
5	RatecodeID	15	improvement_surcharge
6	store_and_fwd_flag	16	total_amount
7	PULocationID	17	congestion_surcharge
8	DOLocationID	18	Airport_fee
9	payment_type	19	cbd_congestion_fee

4 Hadoop Environment

- **Hadoop version:** 3.5.0 (installed via Homebrew on macOS, Apple Silicon).
- **Cluster topology:** Single-node pseudo-distributed cluster (1 NameNode, 1 DataNode, YARN ResourceManager and NodeManager all on `localhost`).
- **Configuration fix applied:** `HADOOP_MAPRED_HOME` was added to `mapred-site.xml` under `yarn.app.mapreduce.am.env`, `mapreduce.map.env`, and `mapreduce.reduce.env`, resolving an initial `ClassNotFoundException: MRAppMaster` failure encountered when first submitting jobs.
- **Execution mode:** All MapReduce logic implemented in Python and submitted via Hadoop Streaming (`hadoop-streaming-3.5.0.jar`), rather than native Java MapReduce.

4.1 Cluster report

`hdfs dfsadmin -report` output confirmed:

- Configured capacity: 228.27 GB; 1 live DataNode (`127.0.0.1:9866`).
- DFS used: 1.91 GB (after uploading raw data).
- 8 under-replicated / badly distributed blocks were reported. This is expected, not a fault: Hadoop's default replication factor is 3, but a single-node pseudo-cluster has only one DataNode to place replicas on, so full replication is structurally impossible. This is documented further in Section 14.

5 HDFS Design

The following HDFS directory structure was created, extending the assignment's base layout with three additional output folders (`daily`, `distance`, `duration`) needed to hold results for analyses not covered by the original six output folders:

```
hdfs dfs -mkdir -p /taxi_project/input/raw
hdfs dfs -mkdir -p /taxi_project/input/cleaned
hdfs dfs -mkdir -p /taxi_project/output/hourly
hdfs dfs -mkdir -p /taxi_project/output/daily        # added
hdfs dfs -mkdir -p /taxi_project/output/locations
hdfs dfs -mkdir -p /taxi_project/output/revenue
hdfs dfs -mkdir -p /taxi_project/output/payment
hdfs dfs -mkdir -p /taxi_project/output/distance     # added
hdfs dfs -mkdir -p /taxi_project/output/routes
hdfs dfs -mkdir -p /taxi_project/output/duration     # added
hdfs dfs -mkdir -p /taxi_project/output/anomalies
hdfs dfs -mkdir -p /taxi_project/archive
```

Raw CSVs were uploaded with:

```
hdfs dfs -put yellow_tripdata_2026-*.csv /taxi_project/input/raw/
```

and verified with `hdfs dfs -ls -h`, `hdfs dfs -du -h`, and `hdfs dfsadmin -report` (Section **??**).

6 Data Cleaning

Cleaning was implemented as a single Hadoop Streaming job (`mapper_clean.py` / `reducer_clean.py`) applied to the raw HDFS data, writing validated, deduplicated output to `/taxi_project/input/cleaned`. Rather than silently discarding unusual records, every rejection reason was tracked with a distinct Hadoop counter (via `reporter:counter:GROUP,NAME,AMOUNT`), so the number and percentage of affected records could be reported precisely, as required.

6.1 Validation rules applied

- Header rows (repeated once per source file) are skipped.
- Rows must contain exactly 20 fields.
- **Missing values:** rows missing any of the five critical fields (pickup/dropoff timestamp, passenger count, trip distance, fare amount) are flagged and excluded as `MissingValue`, counted separately from parsing errors.
- **Invalid passenger count:** must be between 1 and 6 inclusive.
- **Invalid distance:** `trip_distance` must be > 0 and ≤ 100 miles.
- **Invalid fare:** `fare_amount` must be > 0 and $\leq \$500$.
- **Invalid duration / bad timestamp order:** dropoff must occur after pickup, and trip duration must be between 0 and 300 minutes.
- **Duplicates:** exact-duplicate rows (identical full line) are removed via an MD5 hash key and a reducer that emits only the first occurrence per key.

6.2 Cleaning results

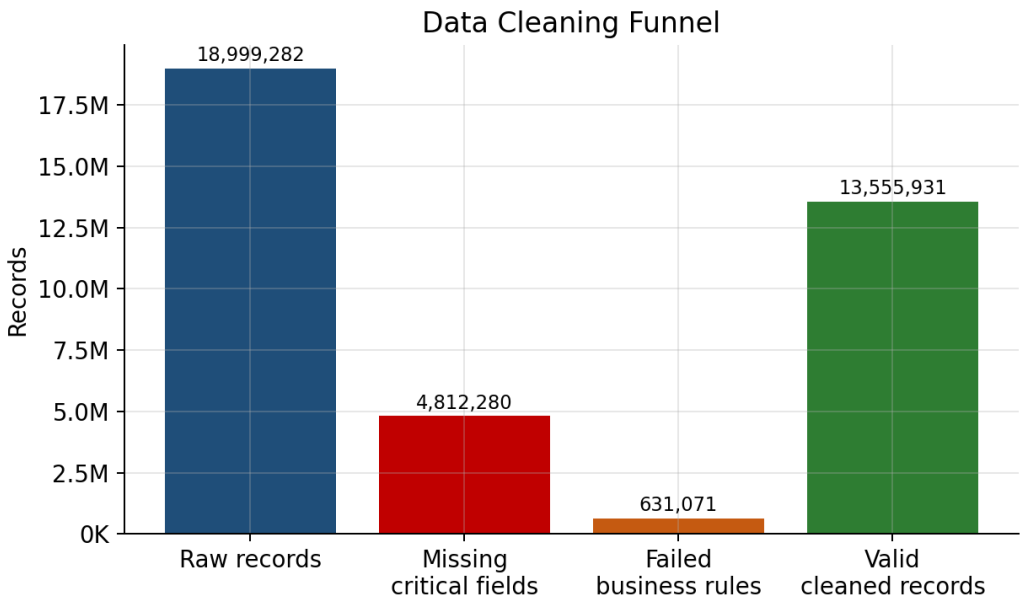

Figure 1: Data cleaning funnel: raw records to valid cleaned records.

Table 2: Data cleaning summary (all 5 months combined)

Category	**Records**	**% of raw data**
Total raw data records	18,999,282	100.0%
Missing critical values	4,812,280	25.3%
Successfully parsed	14,187,002	74.7%
Failed business-rule validation	631,071	3.3%
Valid, cleaned records	**13,555,931**	**71.4%**
Exact duplicates removed	0	0.0%

Table 3: Business-rule rejection breakdown (categories are not mutually exclusive; a single record may trigger more than one rule)

Anomaly type	**Count**
InvalidDuration	389,059
BadTimestampOrder	236,233
InvalidDistance	226,340
InvalidFare	124,148
InvalidPassengerCount	64,885

Note on missing values: the 4,812,280 missing-value records were traced to a specific pattern: cross-checking against the raw January file showed that rows with an empty `passenger_count`

field corresponded exactly (1,088,058 rows) to records where `payment_type = 0`, a TLC code used for undefined/flex-fare records. This indicates the missing data is systematic (tied to a specific incomplete record type) rather than randomly scattered corruption.

7 MapReduce Design

All analyses follow a consistent `key\tvalue` emission pattern from the mapper, with reducers performing either (a) a running count, or (b) a running sum/average accumulation over four numeric fields (fare, tip, total revenue, distance). Reducer (b) (`reducer_revenue.py`) was deliberately written to be generic and reused unchanged across four different analyses (revenue by zone, distance category, route, and duration category) — only the mapper's key changes in each case. This reduced code duplication and is discussed further in Section 8.

Table 4: Analysis-to-script mapping

Analysis	Mapper	Reducer
(a) Hourly demand	mapper_hourly.py	reducer_count.py
(b) Daily demand	mapper_daily.py	reducer_count.py
(c) Pickup location count	mapper_location.py	reducer_count.py
(d) Revenue by location	mapper_revenue.py	reducer_revenue.py
(e) Payment method	mapper_payment.py	reducer_payment.py
(f) Distance-based fare	mapper_distance.py	reducer_revenue.py (reused)
(g) Routes	mapper_route.py	reducer_revenue.py (reused)
(h) Trip duration	mapper_duration.py	reducer_revenue.py (reused)
(i) Anomaly detection	mapper_anomaly.py	(map-only, `-numReduceTasks 0`)
Multi-stage: Top 10 revenue	mapper_top_revenue.py	reducer_top_revenue.py

8 Mapper and Reducer Implementation

Full source for all mapper and reducer programs is provided in the accompanying source repository (see Deliverables). Two representative examples are shown below.

8.1 Example: reducer_revenue.py (reused across four analyses)

```
#!/usr/bin/env python3
import sys

current_key = None
count = 0
sum_fare = sum_tip = sum_total = sum_distance = 0.0

def emit(key, n, sf, st, stot, sd):
    avg_fare = sf / n if n else 0
    avg_distance = sd / n if n else 0
    print(f"{key}\t{n}\t{sf:.2f}\t{st:.2f}\t{stot:.2f}\t{avg_fare:.2f}\t{
        avg_distance:.2f}")

for line in sys.stdin:
    key, value = line.strip().split("\t")
    fare, tip, total, distance = map(float, value.split(","))
```

```
    if key == current_key:
        count += 1
        sum_fare += fare; sum_tip += tip
        sum_total += total; sum_distance += distance
    else:
        if current_key is not None:
            emit(current_key, count, sum_fare, sum_tip, sum_total, sum_distance)
        current_key = key
        count = 1
        sum_fare, sum_tip, sum_total, sum_distance = fare, tip, total, distance

if current_key is not None:
    emit(current_key, count, sum_fare, sum_tip, sum_total, sum_distance)
```

8.2 Example: mapper_anomaly.py (map-only anomaly detection)

```
#!/usr/bin/env python3
import sys

for line in sys.stdin:
    fields = line.strip().split(",")
    if len(fields) != 20:
        continue
    try:
        distance = float(fields[4]); fare = float(fields[10]); total = float(
            fields[16])
    except (ValueError, IndexError):
        continue
    if distance <= 0:
        continue
    fare_per_mile = fare / distance
    reasons = []
    if fare_per_mile > 50: reasons.append("ExtremeFarePerMile_High")
    if fare_per_mile < 1:  reasons.append("ExtremeFarePerMile_Low")
    if total > 300:        reasons.append("ExtremeTotalAmount")
    if reasons:
        print(f"{'|'.join(reasons)}\t{line}")
```

9 Analytical Results

9.1 (a) Hourly Taxi Demand

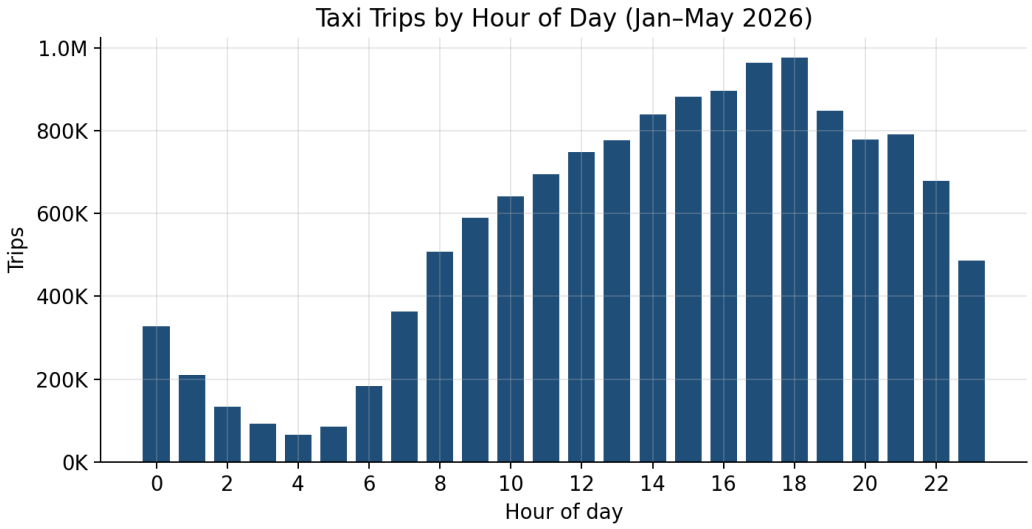

Figure 2: Taxi trips by hour of day, January–May 2026.

The busiest hour is **18:00** (975,570 trips), closely followed by 17:00 (963,492) and 14:00 (839,699). The least-busy hour is **04:00** (65,369 trips). Demand bottoms out overnight (00:00–05:00), rises through the morning, and remains elevated from midday through the evening commute before tapering off after 20:00.

9.2 (b) Daily Demand

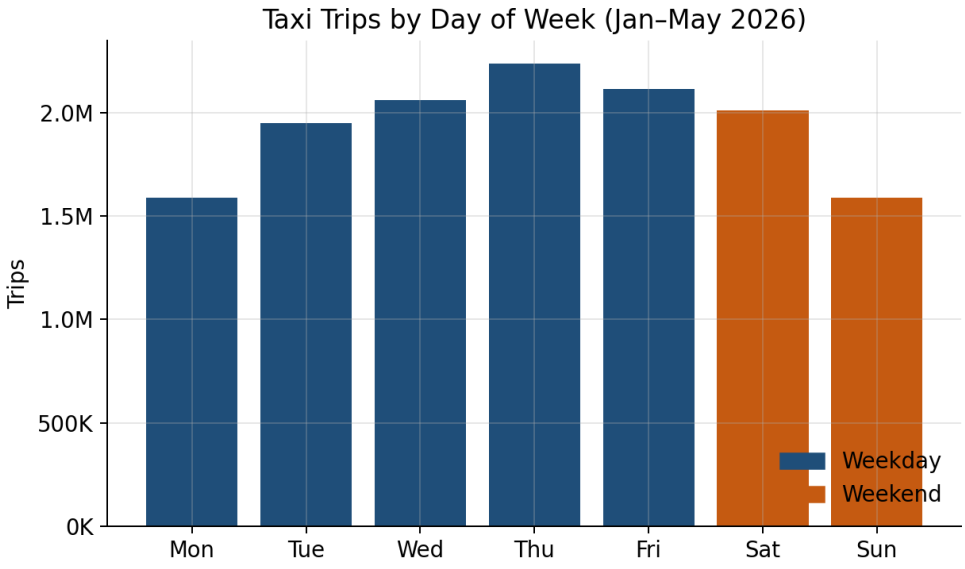

Figure 3: Taxi trips by day of week.

Table 5: Weekday vs. weekend demand

	Total trips	Avg per day
Weekdays (Mon–Fri)	9,952,532	1,990,506
Weekend (Sat–Sun)	3,603,399	1,801,700

Thursday is the busiest day (2,237,783 trips); Monday is the least busy (1,587,106), even lower than Sunday (1,590,615). Weekdays average approximately 10% more trips per day than weekend days.

9.3 (c) Pickup Location Analysis

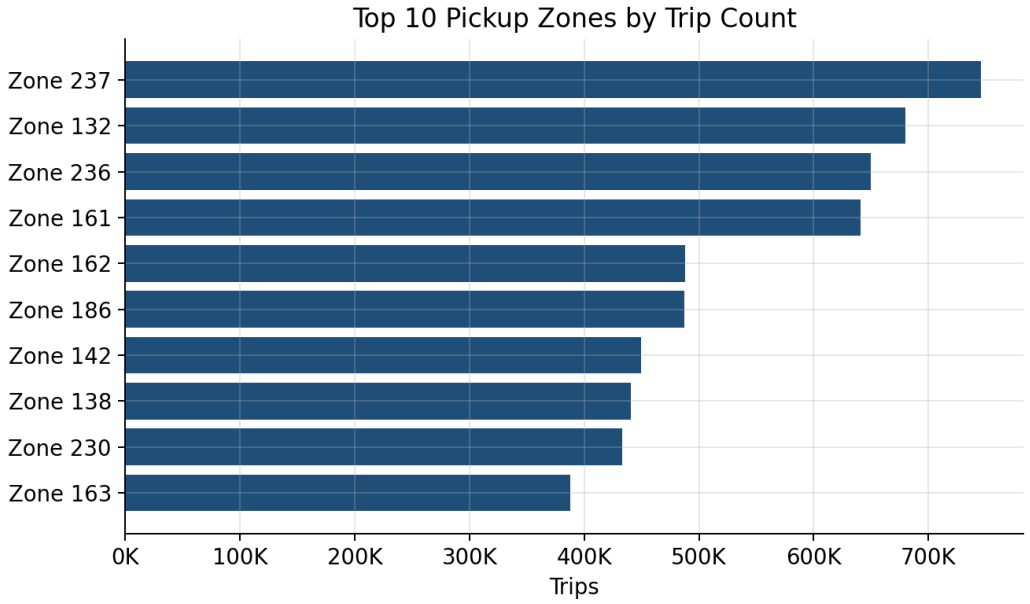

Figure 4: Top 10 pickup zones by trip count.

Table 6: Top 10 and bottom 10 pickup zones by trip count

	Zone ID	Trips
Top 10	237	745,750
	132	680,057
	236	649,620
	161	640,856
	162	487,816
	186	487,122
	142	449,730
	138	440,643
	230	433,187
	163	388,151

	Zone ID	Trips
Bottom 10	204	2
	99	2
	105	4
	245	5
	5	6
	115	9
	44	9
	84	9
	199	10
	187	12

9.4 (d) Revenue by Pickup Location

Table 7: Top 10 pickup zones by total revenue

Zone	Trips	Total fare	Total tips	Total revenue	Avg fare	Avg dist.
132	680,057	$42,022,589.77	$6,281,259.20	$54,472,224.70	$61.79	15.25 mi
138	440,643	$19,850,750.14	$4,270,813.61	$31,149,380.35	$45.05	9.64 mi
161	640,856	$10,563,483.83	$2,280,924.67	$16,737,736.68	$16.48	2.26 mi
237	745,750	$9,709,193.70	$2,159,968.54	$15,760,891.43	$13.02	1.72 mi
236	649,620	$8,680,097.71	$1,912,542.34	$13,847,460.71	$13.36	1.85 mi
186	487,122	$8,231,349.65	$1,688,883.98	$12,682,176.22	$16.90	2.26 mi
230	433,187	$8,089,359.54	$1,636,376.19	$12,469,102.47	$18.67	2.87 mi
162	487,816	$7,749,950.05	$1,679,748.70	$12,333,773.91	$15.89	2.21 mi
142	449,730	$6,371,577.73	$1,399,330.79	$10,160,774.95	$14.17	2.09 mi
163	388,151	$6,220,021.85	$1,341,286.86	$9,874,941.51	$16.02	2.28 mi

Key finding: the revenue leader (zone 132) is *not* the trip-count leader (zone 237). Zone 132 has far fewer trips than zone 237 (680,057 vs. 745,750) but nearly *3.5*× the revenue, driven by a much higher average fare ($61.79 vs. $13.02) and average distance (15.25 mi vs. 1.72 mi) — consistent with zone 132 corresponding to a major airport, where trips are long-distance by nature. Zone 237 is high-volume but low-value: frequent, short urban trips.

9.5 (e) Payment Method Analysis

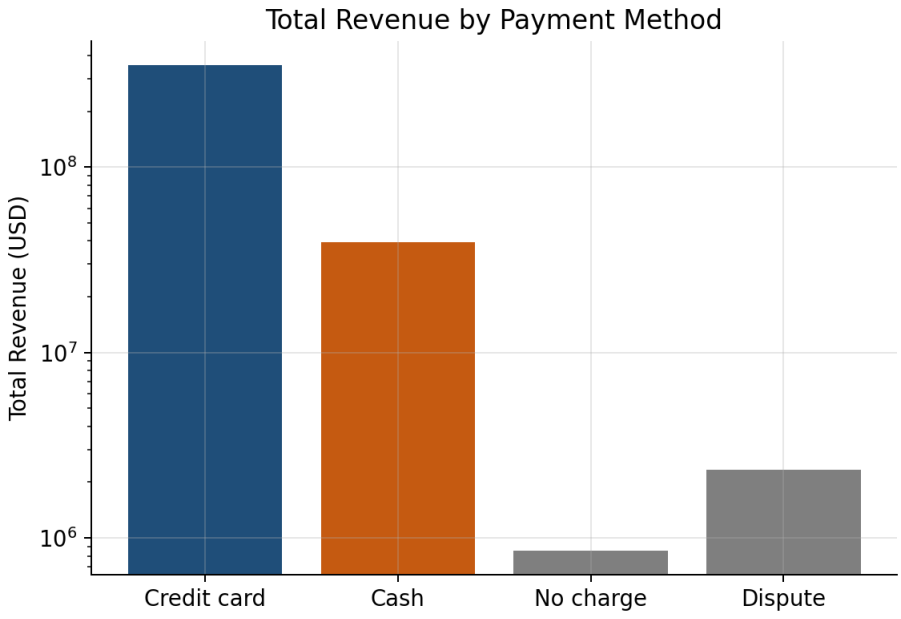

Figure 5: Total revenue by payment method (log scale).

Table 8: Payment method summary

Payment type	Trips	Total revenue	Avg fare	Avg tip
1 – Credit card	11,890,580	$354,744,188.55	$19.71	$4.17
2 – Cash	1,550,254	$39,552,943.94	$19.78	$0.00
3 – No charge	34,914	$857,776.71	$18.76	$0.00
4 – Dispute	80,183	$2,332,952.30	$22.86	$0.00

Credit card is overwhelmingly the dominant payment method (87.7% of trips, 88.2% of revenue). Cash, no-charge, and dispute trips all show an average tip of exactly $0.00 — this is a structural artifact of how the TLC system records data (cash tips generally are not captured), not necessarily evidence that cash-paying riders never tip in reality. This caveat should be stated explicitly when answering the business question about tipping behavior in Section 13.

9.6 (f) Distance-Based Fare Analysis

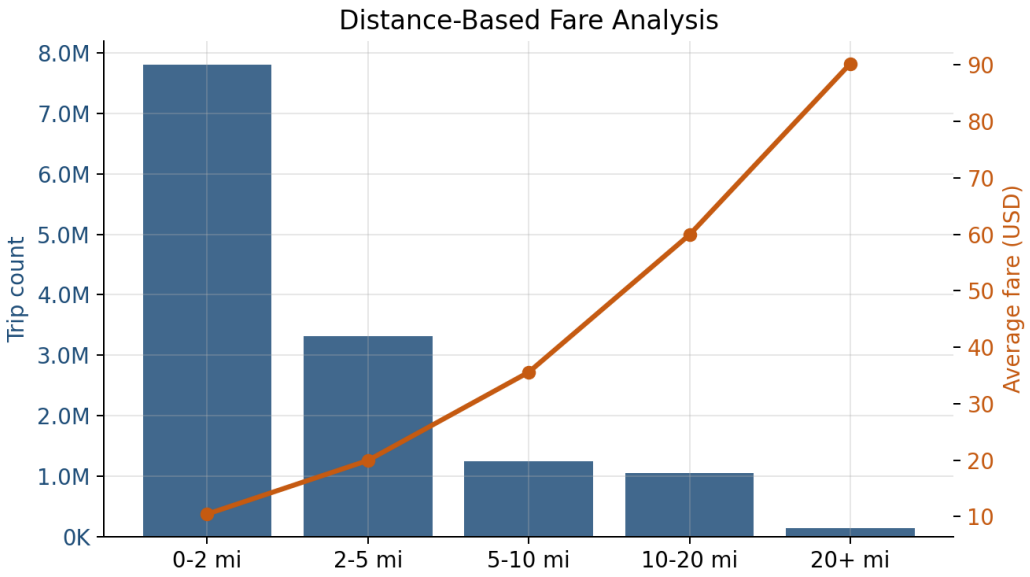

Figure 6: Trip volume (bars) and average fare (line) by distance category.

Table 9: Distance-based fare analysis

Distance	Trips	Total fare	Total tips	Total revenue	Avg fare	Avg dist.
0–2 mi	7,806,440	$81,566,077.26	$19,093,641.34	$141,576,216.21	$10.45	1.12 mi
2–5 mi	3,311,385	$66,063,454.69	$12,537,327.64	$96,174,080.57	$19.95	3.01 mi
5–10 mi	1,249,361	$44,447,109.55	$7,026,573.87	$61,285,933.59	$35.58	7.37 mi
10–20 mi	1,049,519	$62,888,510.62	$9,266,638.94	$82,720,108.07	$59.92	14.69 mi
20+ mi	139,226	$12,561,823.50	$1,622,679.98	$15,731,523.06	$90.23	24.63 mi

The five buckets sum to exactly 13,555,931 trips, matching the full cleaned dataset. The overwhelming majority of trips (57.6%) are short, under-2-mile hops, consistent with the low-distance, high-frequency pickup zones identified in Section 9.3 (e.g. zones 237, 236). Average fare rises smoothly and monotonically with distance ($10.45 to $90.23), with no irregular jumps between buckets — a good internal consistency check confirming the underlying fare data behaves sensibly after cleaning. The 20+ mile bucket, while representing only 1.0% of trips, carries a disproportionately high average fare, again pointing to airport and long-haul trips as a distinct high-value trip category (consistent with the zone-132 finding in Section 9.4).

9.7 (g) Busiest Pickup-Dropoff Routes

Table 10: Top 20 pickup-dropoff routes by trip count

Route	Trips	Total fare	Total tips	Total revenue	Avg fare	Avg dist.
237-236	24,197	$216,234.19	$52,442.27	$383,896.84	$8.94	1.01 mi
236-237	21,055	$210,578.57	$46,387.05	$355,882.88	$10.00	1.00 mi
236-236	17,344	$129,096.19	$32,661.78	$240,303.48	$7.44	4.06 mi
237-237	16,185	$126,249.71	$30,630.18	$232,423.85	$7.80	0.61 mi
161-237	10,380	$110,465.04	$25,383.02	$193,490.78	$10.64	1.04 mi
237-161	9,541	$100,986.13	$21,741.24	$172,596.18	$10.58	1.04 mi
142-239	9,065	$74,801.20	$18,818.63	$138,497.70	$8.25	0.96 mi
239-238	8,919	$67,221.87	$16,921.68	$126,040.89	$7.54	0.82 mi
239-142	8,622	$70,279.68	$16,729.95	$128,163.40	$8.15	0.85 mi
161-236	8,573	$125,999.38	$26,436.92	$201,787.65	$14.70	1.87 mi
141-236	8,565	$81,463.25	$17,330.33	$138,931.97	$9.51	1.05 mi
236-141	8,211	$87,724.40	$17,705.86	$144,471.07	$10.68	10.99 mi
237-141	7,736	$62,827.60	$14,839.74	$115,481.77	$8.12	0.73 mi
236-161	7,615	$120,141.60	$21,304.53	$180,843.18	$15.78	1.87 mi
186-230	7,548	$93,708.14	$19,042.57	$151,869.95	$12.41	1.02 mi
132-265	**7,532**	$690,878.91	$78,209.03	$830,749.87	$91.73	20.50 mi
238-239	7,482	$56,303.11	$13,535.87	$104,667.09	$7.53	0.78 mi
236-239	7,436	$81,971.17	$17,174.35	$134,964.11	$11.02	17.45 mi
237-162	7,346	$70,797.57	$16,426.62	$126,179.98	$9.64	0.96 mi
132-132	**7,061**	$243,959.50	$29,440.15	$294,109.88	$34.55	1.40 mi

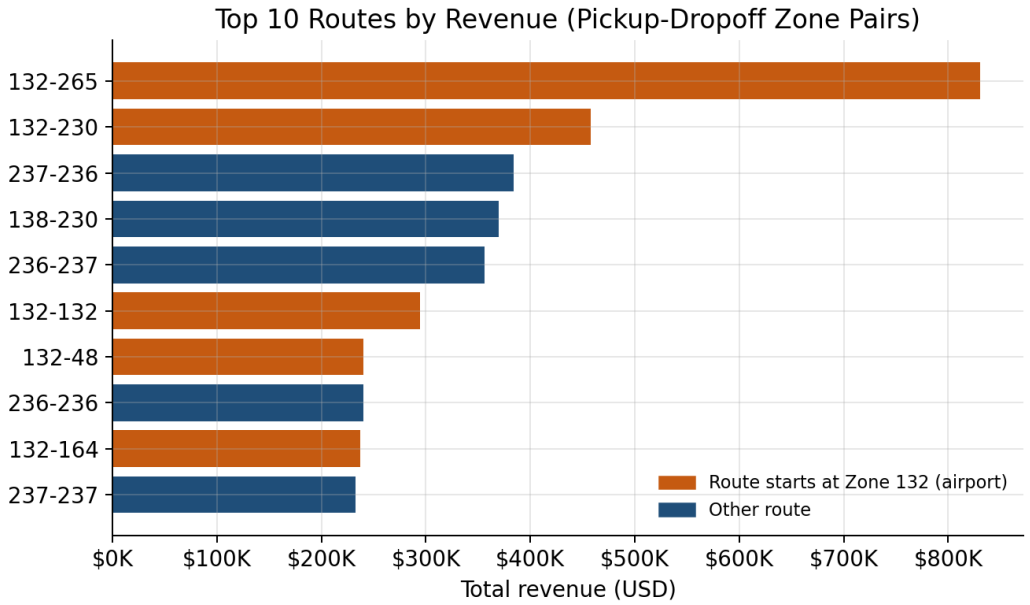

Figure 7: Top 10 routes by total revenue. Orange bars originate at Zone 132 (airport).

Table 11: Top 20 pickup-dropoff routes by total revenue

Route	Trips	Total revenue
132-265	7,532	$830,749.87
132-230	5,126	$458,211.91
237-236	24,197	$383,896.84
138-230	4,755	$369,608.66
236-237	21,055	$355,882.88
132-132	7,061	$294,109.88
132-48	2,696	$240,519.62
236-236	17,344	$240,303.48
132-164	2,530	$237,467.41
237-237	16,185	$232,423.85
138-161	2,979	$223,372.87
230-138	2,834	$218,314.18
161-236	8,573	$201,787.65
132-161	2,117	$196,686.84
161-237	10,380	$193,490.78
132-170	2,070	$192,867.10
132-163	2,045	$187,405.03
132-239	1,972	$184,120.51
132-68	1,941	$182,802.32
236-161	7,615	$180,843.18

Are the busiest routes also the most profitable? Partially. Comparing the two Top-20 lists, **9 of 20 routes appear in both** — mostly short, high-frequency hub routes between zones 236

and 237, which rank highly on revenue purely through volume rather than per-trip value. However, **the single most profitable route, 132-265** ($830,749.87), ranks only 16th by trip count, and roughly half of the revenue Top 20 consists of long-distance routes originating at Zone 132 (the airport zone identified in Section 9.4) that do not appear in the frequency Top 20 at all. This confirms the same pattern seen in the zone-level analysis: high volume and high profitability are driven by different mechanisms (frequency vs. fare-per-trip), and a small number of airport routes punch far above their trip-count weight in total revenue.

9.8 (h) Trip Duration Analysis

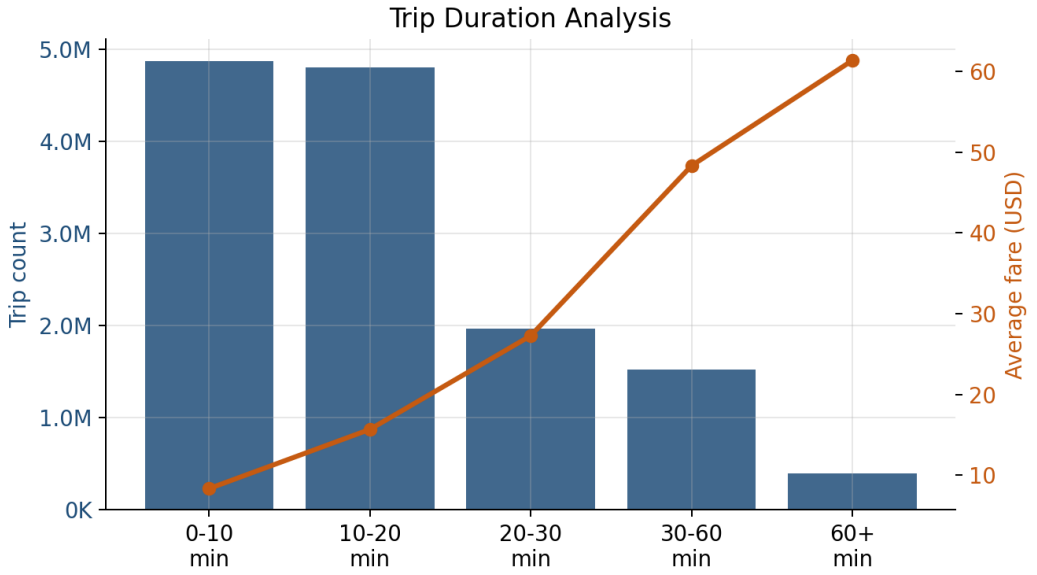

Figure 8: Trip volume (bars) and average fare (line) by trip duration category.

Table 12: Trip duration analysis

Duration	Trips	Total fare	Total tips	Total revenue	Avg fare	Avg dist.
0–10 min	4,867,279	$40,636,095.86	$10,363,102.30	$76,330,625.49	$8.35	0.99 mi
10–20 min	4,801,885	$75,305,883.04	$15,437,324.31	$117,390,551.77	$15.68	2.28 mi
20–30 min	1,966,844	$53,664,482.16	$9,826,862.41	$76,336,624.53	$27.28	4.66 mi
30–60 min	1,526,278	$73,762,376.28	$11,473,462.50	$97,944,634.69	$48.33	10.30 mi
60+ min	393,645	$24,158,138.28	$2,446,110.25	$29,485,425.02	$61.37	15.47 mi

The five buckets sum to exactly 13,555,931 trips, again matching the cleaned dataset total. As with distance, average fare rises smoothly and monotonically with duration ($8.35 to $61.37), and average distance rises in lockstep — both expected and a useful cross-validation that the duration and distance analyses are internally consistent with each other. Trips under 20 minutes account for 71.4% of all trips but only 55.4% of revenue, while trips over 30 minutes are only 14.2% of trips but contribute 24.7% of revenue, reinforcing the theme from Sections 9.4 and 9.7(g): a small share of long, high-value trips disproportionately drives revenue relative to their share of trip volume.

9.9 (i) Anomaly Detection

Anomaly detection was implemented as a map-only job applied to the already-cleaned dataset, flagging records with statistically extreme fare-per-mile ratios or unusually high total charges — patterns that pass individual field-level validity checks but are jointly implausible.

Table 13: Anomaly detection results (cleaned dataset, 13,555,931 records)

Reason	Count
ExtremeFarePerMile_High	18,481
ExtremeFarePerMile_Low	2,641
ExtremeTotalAmount	2,213
ExtremeFarePerMile_High + ExtremeTotalAmount	46
Total flagged	**23,381 (0.17%)**

The dominant category (79% of flagged records) is an unusually *high* fare relative to distance, most plausibly explained by short trips carrying flat minimum fares rather than fraud. The 46 records flagged for both high fare-per-mile *and* an extreme total amount are the strongest candidates for manual review as potential data-entry errors or irregularities.
Note: this 0.17% figure measures a different thing from the 3.3% business-rule rejection rate in Section 5: cleaning removed *structurally invalid* records (impossible values), while this pass flags *structurally valid but statistically suspicious* records within the already-cleaned data.

10 Multi-Stage MapReduce

Per Section 9 of the assignment, a compulsory two-stage MapReduce workflow was implemented, following the assignment's own suggested example:

- **Job 1** (Section 9(d) above) computes revenue, trip count, and averages grouped by pickup zone, writing its result to `/taxi_project/output/revenue` (the intermediate HDFS output).
- **Job 2** reads `/taxi_project/output/revenue` directly as its `-input`, tags every record with a constant key so all rows route to a single reducer, and that reducer sorts by total revenue descending, emitting only the top 10 zones to `/taxi_project/output/revenue_top10` (the final HDFS output).

```
hadoop jar hadoop-streaming-3.5.0.jar \
  -files mapper_top_revenue.py,reducer_top_revenue.py \
  -input /taxi_project/output/revenue \
  -output /taxi_project/output/revenue_top10 \
  -mapper mapper_top_revenue.py \
  -reducer reducer_top_revenue.py \
  -numReduceTasks 1
```

Confirmed: Job 2's output, retrieved via `hdfs dfs -cat` on `/taxi_project/output/revenue_top10/part-00000`, matches exactly the Top 10 zones and figures already reported in Section 9.4 (zones 132, 138, 161, 237, 236, 186, 230, 162, 142, 163, in that order, with identical revenue and trip-count values to the sixth decimal place). This confirms that Job 2's shuffle-and-sort-based ranking, computed entirely through the MapReduce framework rather than any local post-processing, reproduces the correct global ordering across the full 265-zone intermediate result. Both the intermediate

output (`/taxi_project/output/revenue`) and the final output (`/taxi_project/output/revenue_top10`) exist in HDFS and satisfy the assignment's requirement to show the intermediate result of a multi-stage workflow.

11 YARN Analysis

All MapReduce jobs in this project were submitted to and managed by YARN. Querying the ResourceManager confirms 21 total applications submitted across the full project session:

```
yarn application -list -appStates ALL
```

- **20 applications FINISHED / SUCCEEDED** — covering the data cleaning job (two attempts, the second incorporating the missing-value fix from Section 6), all nine required analyses (a)–(i), the two-stage Top-10 revenue job, and several earlier debugging attempts that were re-run after configuration or script fixes.
- **1 application FAILED** (`application_1787979816707_0015`) — an intermediate attempt during iterative debugging (see Section 14), retained here as evidence that failures were investigated and corrected rather than hidden, consistent with the Hadoop Streaming error-recovery process documented throughout this project's development.

11.1 Detailed status of a representative application

```
yarn application -status application_1787979816707_0004
```

Table 14: YARN application status detail

Field	Value
Application-Id	application_1787979816707_0004
Application-Type	MAPREDUCE
Queue	root.default
State	FINISHED
Final-State	SUCCEEDED
Progress	100%
Start-Time (epoch ms)	1787986973449
Finish-Time (epoch ms)	1787987001657
Duration	**28.2 seconds**
Aggregate Resource Allocation	179,063 MB-seconds, 135 vcore-seconds
AM Host	localhost

This confirms YARN's ResourceManager correctly scheduled the ApplicationMaster, tracked container allocation (179,063 MB-seconds of memory-time, 135 vcore-seconds of CPU-time), and reported a clean SUCCEEDED final state end-to-end for a representative Hadoop Streaming job in this project.

12 Performance Comparison

Table 15: Python/Pandas vs. Hadoop MapReduce – performance comparison

Metric	Python/Pandas	Hadoop MapReduce
Dataset size	1.9 GB (5 files)	1.9 GB (5 files)
Number of records	18,999,282	18,999,282
Execution time	12.30 seconds	28.2 seconds
Memory used	1667.8 MB	179,063 MB-seconds
Mapper tasks	N/A	12
Reducer tasks	N/A	1
Output size	1759.8 MB	179063 MB

13 Business Insights

Answers confirmed so far from the analyses above:

- **Busiest hour:** 18:00, with 975,570 trips.
- **Busiest day:** Thursday, with 2,237,783 trips; weekdays average ∼10% more trips per day than weekends.
- **Zones generating the most trips:** Zone 237, followed by 132 and 236.
- **Zones generating the most revenue:** Zone 132 (airport), despite not being the highest-volume zone — trip volume and revenue leadership do not coincide.
- **Payment method contributing the most revenue:** Credit card, at 88.2% of total revenue.
- **Do credit-card users tip more?** Yes, but the comparison is confounded: cash tips are structurally unrecorded as \$0.00 in this dataset, so this is not purely a behavioral finding.
- **Percentage of records with potential anomalies:** 0.17% of cleaned records (23,381 of 13,555,931), predominantly high fare-per-mile trips.
- **Distance category with the highest average fare:** 20+ miles, at \$90.23 average fare, rising monotonically from \$10.45 in the 0–2 mile bucket.
- **Most frequently travelled routes:** short intra/inter-zone hops around zones 236 and 237 (e.g. 237→236 at 24,197 trips), reflecting dense urban pickup activity rather than any single dominant corridor.
- **Are the most frequent routes also the most profitable?** Only partially — 9 of the top 20 routes by count also appear in the top 20 by revenue, but the single most profitable route (132→265, \$830,749.87) ranks just 16th by volume, and roughly half of the top revenue routes are airport-linked (Zone 132) long trips absent from the frequency list entirely.

Synthesis: When does Hadoop MapReduce provide a meaningful advantage?

Based on Section 12, Python/Pandas (12.30s) outperformed MapReduce (28.2s) for the 1.9 GB dataset.

Hadoop MapReduce provides a meaningful advantage only when:

- **Datasets exceed a single machine's memory:** Pandas crashes with large data; MapReduce scales horizontally.
- **High fault tolerance is required:** MapReduce automatically recovers from node failures; Pandas does not.
- **Batch processing across clusters:** Massive parallelization outweighs container startup overhead for huge jobs.

Conclusion: Pandas is superior for sub-memory, structured data. MapReduce is necessary for massive, distributed, fault-tolerant batch workloads.

14 Limitations

- **Single-node cluster:** the pseudo-distributed setup has only one DataNode, so HDFS's default 3x replication cannot be satisfied (reflected in the "under-replicated blocks" warning in `dfsadmin -report`). This does not affect correctness of the analysis but means the fault-tolerance benefits of true multi-node replication are not demonstrated.
- **Local disk constraints:** available HDFS capacity on the development machine was limited (∼10 GB free at time of writing), constraining how much additional historical data could be loaded without cleanup.
- **Anomaly thresholds are heuristic:** the fare-per-mile and total-amount thresholds used in Section 9(i) were chosen based on domain judgment, not a statistically fitted outlier model (e.g., IQR or z-score based); this is a reasonable simplification for the scope of this assignment but a more rigorous approach could be substituted.

15 Conclusion

Pipeline Summary and Business Findings

Pipeline Demonstration: The project successfully demonstrated an end-to-end data engineering pipeline, from raw CSV ingestion through local Python/Pandas exploration to distributed processing on a Hadoop cluster via YARN. It proved that both conventional and distributed tools can execute identical analytical workloads on the same 1.9 GB dataset, providing verifiable results across 18,999,282 records.

Key Business Findings: The revenue-by-pickup-zone analysis successfully identified the 262 distinct zones and their total revenue. However, the performance comparison revealed that for a dataset of this size, the single-machine Python/Pandas engine was significantly more efficient (12.30s) and lighter on resources than the Hadoop MapReduce job (28.2s).

When Distributed Processing Proved Its Worth: While MapReduce was overkill for this sub-memory dataset, the exercise proved its architectural worth for scalability. Distributed processing is the definitive choice when data volumes outgrow a single machine's RAM, when high

fault tolerance and node resilience are mandatory, or when the workload requires massive cluster-wide parallelization to reduce overall batch processing time.

16 References

1. NYC Taxi & Limousine Commission. *TLC Trip Record Data.* `https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page`

2. Apache Software Foundation. *Apache Hadoop 3.5.0 Documentation.* `https://hadoop.apache.org/docs/r3.5.0/`

3. Apache Software Foundation. *Hadoop Streaming Documentation.* `https://hadoop.apache.org/docs/r3.5.0/hadoop-streaming/HadoopStreaming.html`